skills_faq

Skills FAQ (HXC-159 T-P8.01.3)

Revision: 1 · **Last modified**: 2026-09-23 · **Status**: current

Every entry below is based on a genuine question this multi-phase build (HXC-159 Phases P4–P8) actually surfaced — traced to the phase report, commit, or test that produced it — never invented generic filler. See `docs/guides/skills_user_manual.md` for the reference manual this FAQ complements.

Q: I installed a skill for the CLI. Why doesn't the TUI see it?

A: Before HXC-159 Phase P5 (T-P5.03), this was a real bug, not a misunderstanding — the CLI resolved its user-level skills directory as `os.UserConfigDir()/helixcode/skills`, while the TUI and desktop front-ends resolved `$HOME/.helix/skills`, two **independent, non-overlapping** directories. A skill installed for one front-end was genuinely invisible to the other. This is fixed: every front-end now scans the same four-tier ordered list via `agent.AllSkillsDirs` (`internal/agent/skill_dirs.go:125`), including BOTH the old and the new user-level locations (tiers 1 and 2). If you are still seeing this on a current build, verify your skill file is under one of the four tiers documented in the user manual §2, not some other directory. Full history: `docs/guides/skills_directory_migration.md`.

Q: Why does the cross-consumer parity gate (`CM-CROSS-CONSUMER-PARITY`) fail?

A: As of this revision, running `skillparity` against the ACTUAL `helix_code` and `helix_agent` capability reports over the same fixture corpus surfaces **3 genuine, currently-unmarked surface asymmetries**: `helix_code`'s own skill loader has NO `capability_declaration` / `capability_enforcement` / `audit_log` surface, while `helix_agent`'s does (via `SkillAuthorizer`, HXC-159 T-P6.03). This is not a bug in the parity gate — it is the gate **working exactly as designed**: it exists to catch real, pre-existing capability drift between the two consumers, and it found one. `helix_code`'s TUI/CLI skill system is a lightweight trigger-regex mechanism (see the manual §3) that does not implement `allowed-tools` enforcement, content-hash provenance pinning, or an audit log the way `helix_agent`'s `internal/skills` package does — that gap is tracked as a follow-up item (HXC-159 T-P5.05.4 already documented it; the parity gate's job was to

prove it is *detectable*, not to close it). If you are extending `helix_code`'s loader to add one of those three capabilities, either implement the matching capability on the `helix_agent` side too, or mark it `OptionalCapability{Key, Reason}` on **both** sides — a one-sided mark is deliberately insufficient (`TestDiff_OneSidedMarkIsNotEnough`).

Q: Why can HelixAgent depend on the skills library, but the library can't depend on HelixAgent?

A: This is decision D-4 in the HXC-159 spec, and it is load-bearing, not arbitrary. The upstream `HelixDevelopment/HelixSkills` submodule's own `helix-deps.yaml` already declares `HelixDevelopment/HelixAgent` as **its** dependency. If HelixAgent's own code then imported the skills library as a Go package, that would invert the edge and create a dependency cycle — and CONST-051(C) (submodule dependency-layout discipline) forbids exactly the nested own-org submodule chain that would result. The mandated fix: **HelixAgent is a client, never an importer** — it consumes skills through a registered external tool source / MCP-style client boundary (`ExternalSkillSource.RegisterWith`, `submodules/helix_agent/internal/skills/external_source.go:98`), never a direct Go import. This is mechanically enforced, not just documented: a module-graph edge gate (HXC-159 T-P6.05) scans HelixAgent's full dependency graph and asserts zero edges into the skills module; the same discipline applies to `helix_code`'s own `internal/skillconformance` package, which independently *re-implements* the shared capability-manifest schema rather than importing it (`git diff --stat -- go.mod go.sum` stays empty in both consumer trees).

Q: I tagged a skill as `third-party` trust tier and it refuses to run. Is this broken?

A: No — this is the C5 sandbox precondition gate working as intended, and it is a genuine, honestly-declared limitation rather than a bug. `EnforceSandboxPrecondition` (`submodules/skills/pkg/skills/sandbox_gate.go`) refuses **every** `third-party`-tier activation while `sandboxReady=false` — which is the real current state of this project, because the only shipped `IsolatedExecutor` implementation is `SkipIsolatedExecutor`, a deliberate no-op stand-in, not a working sandbox. Before this gate was landed (HXC-159 T-P7.05.5), the refusal existed only as a doc comment on `VendoredTierEnforced` stating "the refusal belongs to the sandbox gate, which does not exist yet" — i.e. the safety claim was aspirational, not enforced. The gate closes that gap: it now genuinely refuses (naming every offending skill) rather than silently allowing a third-party skill to run with no isolation. If your workflow needs a skill to actually execute today, ship it in the

`vendored` or `built-in` trust tier instead — those do not require a real sandbox executor — and track the real sandbox implementation as separate upstream work.

Q: Where does the `explain-arch` example skill actually live, and can I use it as a template?

A: `.helix/skills/explain-arch.md` in this repository's root — it is a real, loaded, working skill (project tier, §2 of the manual), not a documentation-only example. Its front-matter trigger is `explain (?P<topic>\w+) architecture`; try `explain ensemble architecture` in the TUI and it will substitute an architecture-explanation prompt with `topic` resolved to `ensemble`. Yes, it is a reasonable template for a new project-tier skill — copy the front-matter shape and adjust the trigger regex and body for your use case.

Q: Does installing a skill for one project affect other projects?

A: Only if you install it at a user-level tier (tier 1 or 2 in the manual's directory table) — those are scanned for **every** project you open with that front-end. A skill installed at the project tier (`.helix/skills/` inside a specific project's root) is scoped to that project only, and — being the highest-precedence tier — will override a same-named user-level or external-tier skill just for that project.

See also

- `docs/guides/skills_user_manual.md`
- `docs/CAPABILITIES.md`
- `docs/guides/skills_directory_migration.md`